
```
TOTAL_DISKS = 5 # More disks means a more difficult puzzle.
```

Another common use of inline comments is to add context about the values of variables when you create them:

```
month = 2 # Months range from 0 (Jan) to 11 (Dec).  
catWeight = 4.9 # Weight is in kilograms.  
website = 'inventwithpython.com' # Don't include "https://" at front.
```

Inline comments should not specify the variable's data type, because this is obvious from the assignment statement, unless it's done in the comment form of a type hint, as described in “Backporting Type Hints with Comments” later in this chapter.

Explanatory Comments

In general, comments should explain why code is written the way it is rather than what the code does or how it does what it does. Even with the proper code style and useful naming conventions covered in Chapters 3 and 4, the actual code can't explain the original programmer's intentions. If you wrote the code, you might even forget details about it after a few weeks. Present You should write informative code comments to prevent Future You from cursing Past You.

For example, here's an unhelpful comment that explains what the code is doing. Rather than hinting at the motivation for this code, it states the obvious:

```
>>> currentWeekWages *= 1.5 # Multiply the current week's wages by 1.5
```

This comment is worse than useless. It's obvious from the code that the `currentWeekWages` variable is being multiplied by 1.5, so omitting the comment entirely would simplify your code. The following would be a much better comment:

```
>>> currentWeekWages *= 1.5 # Account for time-and-a-half wage rate.
```

This comment explains the intention behind this line of code rather than repeating how the code works. It provides context that even well-written